JAVA DEVELOPMENT

Complete Domain Learning Guide

A practical guide to the JVM, Java syntax, object-oriented design, collections, exceptions, generics, streams, backend architecture, testing, and build automation.

How to use this guide
Type the examples into small Java files, predict their output, compile and run them, then answer the module checkpoints before checking the answers. The final assessment helps you identify topics to revisit.

Assessment target: 70% — at least 14 correct answers out of 20. You can retake the assessment as needed.

Module 1 — Java & the JVM

How Java code becomes something that runs anywhere

1.1 From source code to execution

Java source code is written in .java files. The compiler javac checks and compiles source into .class files containing bytecode. The Java Virtual Machine (JVM) loads classes, verifies bytecode, and executes it. Frequently used code may be compiled by a just-in-time (JIT) compiler into native machine code. This design lets the same bytecode run on different platforms that provide compatible JVM implementations.

```
// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}

// Commands:
// javac Hello.java
// java Hello
```

Key takeaway: The source file is compiled with javac; run the class by its class name, without adding .class to the java command.

1.2 JVM, JRE, and JDK

The JVM is the engine that executes bytecode. The Java Runtime Environment (JRE) traditionally refers to the JVM plus runtime libraries and supporting files needed to run applications. The Java Development Kit (JDK) includes the compiler and development tools along with runtime components. Modern Java distributions may package these pieces differently, but the roles remain useful to understand.

```
JDK = tools for developing + runtime; JVM = bytecode execution engine; runtime libraries provide Java APIs.
```

Key takeaway: Install a JDK to compile Java code. Check installation with java -version and javac -version.

1.3 Bytecode and portability

Bytecode is an intermediate instruction format, not ordinary source code and not platform-specific native machine code. Portability depends on the target machine having a compatible JVM and required libraries. Native dependencies, operating-system assumptions, and incompatible Java versions can still prevent a program from running unchanged.

```
Compile once to produce a class file, then run it on another supported system with a compatible Java runtime.
```

Key takeaway: “Write once, run anywhere” describes the goal of the platform, not a guarantee that every program has no environment dependencies.

1.4 Memory and garbage collection

Objects are allocated in managed memory. Garbage collection reclaims heap memory for objects that are no longer reachable from live references. Developers generally do not manually free ordinary Java objects, but they should still close files, sockets, and database connections. Garbage collection does not guarantee that resources are closed promptly, and memory leaks can still occur when references to unused objects are retained.

```
String message = new String("hello");
message = null; // the object may become eligible for GC
```

Key takeaway: Eligibility for collection does not mean collection happens immediately. Use try-with-resources for closeable resources rather than relying on garbage collection.

Module checkpoint

What produces .class bytecode from .java source?

Answer: The Java compiler, javac.

What is the JVM responsible for?

Answer: Loading, verifying, and executing bytecode, while managing runtime services such as memory.

Does garbage collection guarantee immediate cleanup of files?

Answer: No. Use explicit resource management such as try-with-resources.

Module 2 — Core Syntax & Types

Primitives, references, and control flow

2.1 Primitive and reference types

Java has eight primitive types: byte, short, int, long, float, double, char, and boolean. A primitive variable holds a primitive value. A reference variable holds a reference to an object or null. Arrays and objects are reference types. Local variables must be assigned before use; fields receive default values such as 0, false, or null.

```
int count = 3;
boolean ready = true;
String name = "Asha";
int[] marks = {80, 91, 88};
```

Key takeaway: Use int for ordinary whole-number calculations; use long when a larger range is required. Use double for common floating-point calculations, but not as an exact money representation.

2.2 Control flow and loops

if/else chooses a path; switch selects among cases; for and enhanced-for loops iterate; while repeats while a condition remains true. Ensure loop conditions eventually change or terminate. Use braces consistently to make scope clear.

```
int total = 0;
for (int mark : marks) {
    if (mark >= 90) total++;
}
System.out.println(total);
```

Key takeaway: Enhanced-for is convenient when you need each element, while an indexed loop is useful when the position matters.

2.3 String immutability and comparison

String objects are immutable: operations such as toUpperCase or concatenation produce a String result rather than changing the original object. Use equals() to compare text content. The == operator compares primitive values or, for references, whether they refer to the same object. String literals may be interned, which can make == appear to work in some cases, but it is not the correct general content comparison.

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

Key takeaway: Use equals() for content equality and Objects.equals(a, b) when either reference may be null.

2.4 Autoboxing and unboxing

Wrapper classes represent primitive values as objects: Integer for int, Double for double, Boolean for boolean, and so on. Autoboxing converts a primitive to its wrapper; unboxing converts a wrapper to a primitive. Collections such as ArrayList use wrapper types because Java generics do not accept primitive type arguments. Unboxing null throws NullPointerException.

```
Integer boxed = 7;  // autoboxing
int value = boxed;  // unboxing
// int n = (Integer) null; // NullPointerException
```

Key takeaway: Be mindful of null wrapper values and avoid unnecessary boxing in performance-sensitive numeric loops.

Module checkpoint

How should two strings be compared by content?

Answer: Use equals(), not ==.

What is autoboxing?

Answer: Automatic conversion from a primitive value to its corresponding wrapper object.

What can happen when a null Integer is unboxed?

Answer: A NullPointerException.

Module 3 — Object-Oriented Java

Classes, interfaces, and contracts

3.1 Classes, objects, constructors, and encapsulation

A class defines fields and methods; an object is an instance of a class. A constructor runs during object creation to initialise state and has the same name as the class with no return type. Encapsulation keeps state controlled by using access modifiers and methods that preserve valid conditions. Prefer private fields and expose only the operations callers need.

```
class Student {
    private final String name;
    private int score;

    Student(String name, int score) {
        this.name = name;
        setScore(score);
    }
    public void setScore(int score) {
        if (score < 0 || score > 100) throw new IllegalArgumentException();
        this.score = score;
    }
    public int getScore() { return score; }
}
```

Key takeaway: A constructor establishes a valid initial state; validation should prevent objects from entering invalid states.

3.2 Interfaces and abstract classes

An interface defines a contract that implementing classes promise to satisfy. It is useful when unrelated classes need to offer the same capability. An abstract class can contain state, constructors, concrete methods, and abstract methods; it cannot be instantiated directly. Use an interface for a capability or contract, and an abstract class when related subclasses share implementation or state.

```
interface Payable {
    double amountDue();
}

class Invoice implements Payable {
    private final double amount;
    Invoice(double amount) { this.amount = amount; }
    public double amountDue() { return amount; }
}
```

Key takeaway: A class can implement multiple interfaces but extend only one class. Design around the contract that callers need.

3.3 Polymorphism

Polymorphism allows code to call the same method on objects of different concrete types. The implementation selected for an overridden instance method is based on the actual object at runtime. This lets systems add new implementations without expanding a large if/else chain in every caller.

```
interface Notifier { void send(String message); }
class EmailNotifier implements Notifier {
    public void send(String message) { System.out.println("Email: " + message); }
}
class SmsNotifier implements Notifier {
    public void send(String message) { System.out.println("SMS: " + message); }
}
// Notifier notifier = new EmailNotifier();
// notifier.send("Welcome");
```

Key takeaway: Program to interfaces when practical. Caller code can depend on Notifier rather than a particular email or SMS class.

3.4 Choosing inheritance or composition

Inheritance expresses an “is-a” relationship: a SavingsAccount is an Account. Composition expresses a “has-a” relationship: a CheckoutService has a PaymentGateway. Composition often makes components easier to replace and test, while inheritance is useful when subclasses truly satisfy the base type's contract.

```
class CheckoutService {
    private final PaymentGateway gateway;
    CheckoutService(PaymentGateway gateway) { this.gateway = gateway; }
}
interface PaymentGateway { void charge(int cents); }
```

Key takeaway: Do not use inheritance merely to reuse a few lines; consider whether the relationship makes conceptual and behavioural sense.

Module checkpoint

When is an interface useful?

Answer: When different classes need to satisfy the same contract.

What is polymorphism?

Answer: Calling the same method through a common type while runtime implementations provide type-specific behaviour.

What relationship does composition express?

Answer: A has-a relationship.

Module 4 — Collections Framework

Lists, sets, maps, and performance

4.1 ArrayList

ArrayList is a resizable array. It provides fast index access and amortized constant-time append. Inserting or removing near the beginning or middle may shift many elements. It permits duplicates and preserves insertion order.

```
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
System.out.println(names.get(0));
```

Key takeaway: Choose ArrayList for most general-purpose list use, especially when reading by index is common.

4.2 HashMap and sets

HashMap stores key-value pairs and usually provides average constant-time get and put when hashes are well distributed. Keys are unique; adding the same key replaces its value. HashSet stores unique elements and is useful for deduplication and membership checks. Do not depend on HashMap or HashSet iteration order.

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Asha", 90);
scores.put("Ravi", 84);
System.out.println(scores.get("Asha"));
Set<String> unique = new HashSet<>(List.of("a", "b", "a"));
```

Key takeaway: Use getOrDefault or containsKey when missing keys need explicit handling. Keys used in hash collections should have stable equals() and hashCode() behaviour.

4.3 LinkedList and operation costs

LinkedList stores elements in linked nodes. Insertion or removal at a known node can be efficient, but finding a position by index requires traversal. Getting the middle element by index is not constant-time. ArrayList often performs better in practice for common list workloads due to memory locality and lower per-element overhead.

```
List<Integer> values = new LinkedList<>();
values.add(10);
values.add(20);
values.add(30);
```

Key takeaway: Choose a collection based on the operations your program actually performs; LinkedList is not automatically faster for insertions when locating the insertion position is also required.

4.4 Comparator and sorting

Comparator defines an ordering outside the class being sorted. It can compare by a field, combine ordering rules, and be expressed with comparator helper methods. Comparisons should be consistent and avoid subtraction that can overflow for integer fields.

```
students.sort(Comparator.comparing(StudentRecord::score)
                        .reversed());
```

Key takeaway: Use Comparator.comparing(...) for readable sort rules; add thenComparing(...) to break ties.

Module checkpoint

Which collection is a common default for indexed list access?

Answer: ArrayList.

What is the average lookup complexity of HashMap?

Answer: Approximately O(1) under typical hash distribution assumptions.

Why use Comparator?

Answer: To define an ordering for sorting objects.

Module 5 — Exception Handling

Checked, unchecked, and how to fail well

5.1 Checked and unchecked exceptions

Checked exceptions are subject to compile-time checking: a method must catch them or declare them with throws. IOException is a common example. Unchecked exceptions are RuntimeException subclasses and their subclasses; the compiler does not force callers to handle them. IllegalArgumentException and NullPointerException are common unchecked exceptions.

```
try {
    Files.readString(Path.of("notes.txt"));
} catch (IOException e) {
    System.err.println("Could not read notes: " + e.getMessage());
}
```

Key takeaway: Catch exceptions where the application can recover, add useful context, or translate them into a meaningful response.

5.2 Try-with-resources

Try-with-resources automatically closes resources that implement AutoCloseable. It is ideal for files, streams, and many database resources. Resources are closed even when an exception occurs, and they are closed in reverse declaration order.

```
try (BufferedReader reader = Files.newBufferedReader(Path.of("notes.txt"))) {
    System.out.println(reader.readLine());
} catch (IOException e) {
    System.err.println("File operation failed");
}
```

Key takeaway: Prefer try-with-resources over manually calling close() in finally blocks for AutoCloseable resources.

5.3 Stack traces and exception boundaries

A stack trace records the method-call path at the time an exception was thrown. The exception type and message are usually shown near the top of the trace, followed by stack frames. Read the first application frame to locate relevant code, then trace the data and call path. Do not log an error and silently continue when the application cannot safely proceed.

```
try {
    int value = Integer.parseInt("not-a-number");
} catch (NumberFormatException e) {
    e.printStackTrace(); // useful during learning/debugging
}
```

Key takeaway: In production, use structured logging and avoid exposing internal traces to end users. Preserve the cause when wrapping exceptions.

Module checkpoint

Give an example of a checked exception.

Answer: IOException.

What does try-with-resources do?

Answer: Automatically closes declared AutoCloseable resources.

What does a stack trace help you find?

Answer: The call path and code location associated with an exception.

Module 6 — Generics & Type Safety

Reusable code that catches errors at compile time

6.1 Generic types and methods

Generics let classes, interfaces, and methods work with different types while preserving compile-time type checks. List communicates that the list contains strings, reducing casts and runtime type errors. Generic type arguments must be reference types, so use Integer rather than int.

```
List<String> languages = new ArrayList<>();
languages.add("Java");
String first = languages.get(0);
```

Key takeaway: Use descriptive type parameters such as T for a general type and K/V for key/value when appropriate.

6.2 Bounded type parameters

A bound restricts the types accepted by a type parameter. For example, T extends Number lets code call Number methods on T. Bounds can use interfaces, and multiple bounds are possible with a class bound first. Bounded generics are useful for reusable algorithms that need a specific capability.

```
static <T extends Number> double asDouble(T value) {
    return value.doubleValue();
}
```

Key takeaway: A bound communicates the capabilities required by the algorithm, not merely a list of allowed classes.

6.3 Wildcards and PECS

A wildcard ? means an unknown type. ? extends T is useful when reading values as T (producer); ? super T is useful when inserting T values (consumer). The mnemonic PECS means Producer Extends, Consumer Super. Wildcards make APIs flexible while preserving type safety.

```
static double sum(List<? extends Number> values) {
    double total = 0;
    for (Number value : values) total += value.doubleValue();
    return total;
}
```

Key takeaway: A List is not a subtype of List. Generics are invariant; wildcards express safe flexibility.

6.4 Type erasure

Java implements most generics through type erasure: compiler-checked type parameters are generally erased or replaced by their bounds in runtime representations, with casts and bridge methods inserted where needed. This supports compatibility with older bytecode. It also means code cannot usually create new T(), test value instanceof List, or overload methods solely by different generic type arguments.

```
// Usually illegal: if (obj instanceof List<String>) { ... }
// A reifiable check is possible:
if (obj instanceof List<?>) { /* ... */ }
```

Key takeaway: Use runtime checks that are actually reifiable, and do not assume a generic type argument is always available at runtime.

Module checkpoint

Why use generics?

Answer: They improve reuse while preserving compile-time type safety.

What does T extends Number permit?

Answer: T must be Number or a subtype of Number, allowing Number methods to be used.

What does PECS mean?

Answer: Producer Extends, Consumer Super.

Module 7 — Streams & Functional Java

Declarative data-processing pipelines

7.1 Lambda expressions and method references

A lambda is a concise implementation of a functional interface, such as Predicate or Function. A method reference points to an existing method and can make simple lambdas clearer. Lambdas are useful for passing behaviour to sorting, filtering, and event-handling APIs.

```
names.forEach(name -> System.out.println(name));
names.forEach(System.out::println);
```

Key takeaway: Use method references when they improve clarity; a lambda is often clearer when it includes non-trivial logic.

7.2 Stream pipelines

A stream pipeline commonly contains a source, zero or more intermediate operations, and a terminal operation. Intermediate operations such as filter and map are lazy: they do not process the data until a terminal operation runs. A stream should generally be consumed once and should not be reused after a terminal operation.

```
List<String> names = List.of("Asha", "Ravi", "Anu", "Mina");
List<String> result = names.stream()
    .filter(name -> name.length() >= 4)
    .map(String::toUpperCase)
    .toList();
System.out.println(result);
```

Key takeaway: filter selects elements; map transforms them; collect/toList or forEach is a terminal operation in the examples shown.

7.3 Reduce and terminal operations

Terminal operations produce a result or side effect, for example count, findFirst, anyMatch, forEach, reduce, or collect. reduce combines elements into one value, while collect builds a mutable result container. Use a clear loop instead of a stream when the pipeline becomes difficult to read or requires complex state.

```
int total = List.of(3, 4, 5).stream()
    .reduce(0, Integer::sum);
```

Key takeaway: Avoid stateful side effects inside stream operations, especially parallel streams; prefer transformations and reductions.

7.4 Optional

Optional represents a value that may be absent. It can make absence explicit in return types and supports methods such as map, filter, orElse, orElseGet, and ifPresent. orElse evaluates its argument immediately, while orElseGet evaluates the supplier only if the Optional is empty.

```
Optional<String> name = Optional.of("Asha");
String shown = name.orElse("Guest");
```

Key takeaway: Optional is commonly useful for return values. It is not a universal replacement for every nullable field or method parameter.

Module checkpoint

When does a stream pipeline normally execute?

Answer: When a terminal operation is invoked.

What does filter do?

Answer: Keeps elements that satisfy a predicate.

What does Optional represent?

Answer: A value that may be absent.

Module 8 — Backend Application Structure

Layering a service so it stays maintainable

8.1 Controller, service, repository, and DTO

A controller handles transport concerns such as HTTP requests and responses. A service layer contains business rules and coordinates use cases. A repository abstracts data access. A DTO (Data Transfer Object) carries data across a boundary without exposing internal domain entities directly. This separation helps each part be tested and changed independently.

```
Request DTO -> Controller -> Service -> Repository -> Database
Response DTO <- Controller <- Service <- Repository result
```

Key takeaway: Keep controllers thin, put business rules in services, and keep storage-specific details behind repository abstractions.

8.2 Dependency injection

Dependency injection supplies a component's dependencies from outside rather than having the component create them internally. Constructor injection makes dependencies explicit, supports immutability, and simplifies unit testing. A dependency injection framework can create and wire objects, but the underlying design idea works without a framework.

```
class ReportService {
    private final ReportRepository repository;
    ReportService(ReportRepository repository) {
        this.repository = repository;
    }
}
```

Key takeaway: A service that receives an interface can be tested with a fake or mock repository without connecting to a real database.

8.3 DTOs and validation

DTOs define the fields accepted or returned at an application boundary. Validate incoming data before applying business operations. Avoid returning database entities directly when that would expose internal fields or couple clients to the persistence model. Map between DTOs and domain objects deliberately.

```
class CreateUserRequest {
    private String email;
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Key takeaway: Boundary validation improves error messages and helps protect business invariants; authentication and authorization are separate requirements.

Module checkpoint

Where should core business rules usually live?

Answer: In the service layer.

What does a repository abstract?

Answer: Data access and persistence details.

Why inject dependencies through a constructor?

Answer: It makes dependencies explicit and simplifies testing.

Module 9 — Testing & Build Tools

Automating the build and proving correctness

9.1 Build tools and dependency management

Maven and Gradle automate compilation, dependency resolution, testing, and packaging. Maven commonly uses pom.xml; Gradle uses build.gradle or build.gradle.kts. Build files document the project and make builds more repeatable than manually assembling JARs and classpaths.

```
# Common Maven commands
mvn test
mvn package

# Common Gradle commands
./gradlew test
./gradlew build
```

Key takeaway: Use the project wrapper (mvnw or gradlew) when provided so teammates use the intended build-tool version.

9.2 Unit tests and assertions

Unit tests verify small behaviours with repeatable inputs and expected outcomes. JUnit is a common Java testing framework. Test normal cases, boundaries, and invalid inputs. Assertions compare actual results with expected results; descriptive test names communicate the behaviour under test.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class DiscountTest {
    @Test void appliesTenPercentDiscount() {
        assertEquals(90.0, 100.0 * 0.9, 0.001);
    }
}
```

Key takeaway: Tests should verify behaviour rather than implementation details, and tests should be independent and deterministic.

9.3 Mocking and build pipelines

A mocking framework such as Mockito creates stand-ins for dependencies. Mocks are useful for verifying interactions or simulating failures at external boundaries, but excessive mocking can make tests brittle. A CI pipeline can compile code, run tests, perform checks, and package the application on every change.

```
// Mockito-style concept:
// when(repository.findById(7L)).thenReturn(Optional.of(user));
// verify(repository).findById(7L);
```

Key takeaway: Use real objects for simple logic and mocks for dependencies that are slow, external, or difficult to control.

9.4 Artifacts and releases

A build artifact is the output that can be stored, shared, or deployed, such as a JAR. A JAR packages compiled classes and resources. Reproducible builds, versioned artifacts, automated tests, and release notes help teams know what was built and which code is running.

```
mvn package
# The JAR is commonly produced under target/
```

Key takeaway: A successful compilation alone does not prove correctness; include automated tests and deployment checks.

Module checkpoint

Name two Java build tools.

Answer: Maven and Gradle.

What is a mock used for?

Answer: Replacing a dependency to isolate behaviour or verify interactions in a test.

What is a JAR?

Answer: A packaged Java archive containing classes and resources.

Practice Assessment — 20 Questions

Choose one best answer per question. Score one point for each correct answer. A score of 14/20 (70%) meets the passing threshold.

1. Which command compiles Hello.java?

A. java Hello.java

B. javac Hello.java

C. jvm Hello.java

D. jar Hello.java

2. What is the JVM's main role?

A. Write Java source code

B. Execute and manage the runtime for bytecode

C. Replace all libraries

D. Convert HTML into Java

3. Which statement compares String contents correctly?

A. a == b

B. a.equals(b)

C. a = b

D. a.compareReference(b)

4. Which is a primitive type?

A. String

B. Integer

C. int

D. ArrayList

5. What can happen when an Integer containing null is unboxed?

A. It becomes 0

B. It throws NullPointerException

C. It becomes false

D. It always compiles to an error

6. Which type defines a contract that classes can implement?

A. Interface

B. Constructor

C. Package

D. Stack trace

7. Which concept lets different implementations respond to the same method call?

A. Boxing

B. Polymorphism

C. Garbage collection

D. Type erasure

8. Which collection is usually best for frequent index-based reads?

A. HashMap

B. HashSet

C. ArrayList

D. TreeMap

9. What is a typical average lookup complexity for HashMap?

A. $O(n^2)$

B. O(n) always

C. O(1)

D. O(log n) always

10. What is the purpose of a Comparator?

A. Close files

B. Define an ordering for sorting

C. Compile source code

D. Catch every exception

11. Which is commonly a checked exception?

A. NullPointerException

B. IllegalArgumentException

C. IOException

D. ArithmeticException

12. What does try-with-resources help guarantee?

A. The resource is closed when the block exits

B. No exception can occur

C. Every object is garbage collected immediately

D. The program always succeeds

13. What is the main benefit of List over a raw List?

A. It makes the list immutable

B. It adds compile-time type safety

C. It sorts values automatically

D. It stores only primitives

14. What does type erasure mean?

A. Source files are deleted

B. Generic type information is generally erased from runtime representations

C. Every object is erased from memory

D. The compiler removes all methods

15. In PECS, what is the usual role of ? extends T?

A. Producer of T values

B. Consumer of T values only

C. A checked exception

D. A terminal stream operation

16. Which is a terminal stream operation?

A. filter

B. map

C. count

D. sorted

17. What does Optional represent?

A. A value that may be absent

B. A Java compiler

C. A file format

D. A thread only

18. Which layer should normally contain business rules?

A. Controller only

B. Service layer

C. DTO

D. Build artifact

19. What is a build tool such as Maven used for?

A. Only edit source files

B. Compile, resolve dependencies, test, and package

C. Replace the JVM

D. Only design user interfaces

20. What is the best role for a mock in a unit test?

A. Replace an external dependency to isolate behaviour

B. Guarantee there are no bugs

C. Replace every assertion

D. Prove production deployment succeeded

ROYSTIAN FOUNDATION

Answer Key with Explanations

1. Correct answer: **B** — javac compiles Java source into bytecode class files.

2. Correct answer: **B** — The JVM loads and executes bytecode and provides runtime services.

3. Correct answer: **B** — equals() checks content equality for String.

4. Correct answer: **C** — int is primitive; Integer is its wrapper class.

5. Correct answer: **B** — Unboxing null attempts to obtain a primitive value and throws NullPointerException.

6. Correct answer: **A** — Interfaces define a contract and can be implemented by classes.

7. Correct answer: **B** — Polymorphism dispatches overridden instance methods according to the runtime object.

8. Correct answer: **C** — ArrayList provides fast index access.

9. Correct answer: **C** — HashMap typically offers average constant-time lookup, subject to hashing and collisions.

10. Correct answer: **B** — Comparator defines how two objects should be ordered.

11. Correct answer: **C** — IOException must be caught or declared by the compiler.

12. Correct answer: **A** — It automatically closes resources that implement AutoCloseable.

13. Correct answer: **B** — Generics catch many wrong-type operations at compile time.

14. Correct answer: **B** — Java generally implements generics through compile-time checking and type erasure.

15. Correct answer: **A** — An extends wildcard is useful for reading values as T.

16. Correct answer: **C** — count consumes the stream and returns a result.

17. Correct answer: **A** — Optional makes possible absence explicit in an API.

18. Correct answer: **B** — The service layer coordinates use cases and owns business logic.

19. Correct answer: **B** — Build tools automate common, repeatable build steps.

20. Correct answer: **A** — Mocks can isolate a unit from external dependencies and verify interactions.

Quick Revision Sheet

Java execution: javac compiles .java to bytecode .class; JVM loads and executes bytecode; JDK includes development tools.

Memory: GC reclaims unreachable objects eventually; close files, sockets, and database resources explicitly.

Types: Primitives store primitive values; references refer to objects; Strings are immutable; use equals() for content.

OOP: Class defines objects; constructor initialises state; interface is a contract; abstract class can share state/implementation.

Collections: ArrayList: fast indexed reads; HashMap: key-value lookup; HashSet: unique elements; LinkedList: traversal by index is slow.

Exceptions: Checked exceptions must be caught or declared; unchecked exceptions are RuntimeException types; try-with-resources closes resources.

Generics: Generics provide compile-time type safety; bounded parameters restrict types; PECS = Producer Extends, Consumer Super.

Streams: filter selects; map transforms; terminal operations execute pipelines; Optional represents possible absence.

Backend layers: Controller handles transport; service contains business rules; repository abstracts persistence; DTO transfers boundary data.

Build and tests: Maven/Gradle automate builds; JUnit asserts behaviour; mocks isolate dependencies; JARs are common build artifacts.

End of guide. Review missed assessment questions by revisiting the related module and writing a small Java example of your own.